

CSC490 Module 1 Exploration

Presented by Lily Yu and Koby Choy
Group 7855 (yuxiangn, choykwan)



Motivation

We want to solve the problem of class imbalance and heatmaps not accurately represented.

What problems are we encountering?

- Class Imbalance
 - Data that are labelled “contain vehicles” is << data that are labelled “do not contain vehicles”
 - Our original model assumes that data are uniformly distributed among the classes
 - might lead to bias -> higher tendency of predicting one class over another, favoring the “no vehicles” class.
- Lost information in heatmaps
 - Data for each detection was not accurately represented
 - Information like headings and bounding box sizes are lost

Describing Methods

Part I: Implementing Focal Loss (Proposed by Lin et al)

$$FL(p_t) = \alpha_t (1 - p_t)^\gamma (-\log(p_t))$$

```
def heatmap_focal_loss(targets: Tensor, predictions: Tensor, gamma: float, alpha: float):  
    # p_t = p when target = 1 and 1-p when target = 0  
    p = predictions  
    p_t = p * targets + (1-p) * (1- targets)  
  
    alpha_t = alpha * targets + (1-alpha) * (1 - targets)  
    focal_loss = alpha_t * (1-p_t)**(gamma) * -(torch.log(p_t))  
  
    return focal_loss.mean()
```

- The focusing parameter gamma dominates the modulating factor $(1 - p_t)^\gamma$ for **focusing training on the hard examples** while down-weighting the easy examples
- The weighing factor alpha **balances the importance** of positive/negative examples
- We tuned these hyperparameters through experimenting with **5 different settings**

Figure 1: Part A predicted heatmap

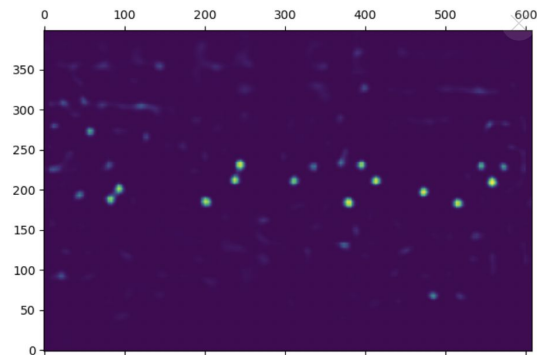
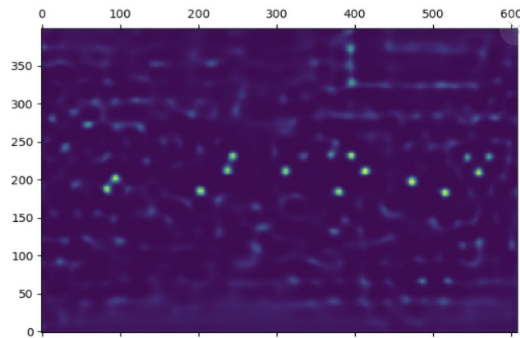


Figure 2: Focal loss predicted heatmap



Describing Methods

Part II: Implementing Rotated Anisotropic Gaussian Kernel for creating heatmap

$$a = \frac{\cos^2(\theta)}{\text{scale}_x} + \frac{\sin^2(\theta)}{\text{scale}_y}$$

$$b = \frac{2 \cos(\theta) \sin(\theta)}{2 \text{scale}_y} - \frac{2 \sin(\theta) \cos(\theta)}{2 \text{scale}_x}$$

$$c = \frac{\cos^2(\theta)}{\text{scale}_y} + \frac{\sin^2(\theta)}{\text{scale}_x}$$

$$e^{-(a(x-x_0)^2 + 2b(x-x_0)(y-y_0) + c(y-y_0)^2)}$$

```
scale_x = 2*(x_size**2) / self._heatmap_norm_scale
```

```
scale_y = 2*(y_size**2) / self._heatmap_norm_scale
```

The scale used for the x-direction and y-direction are based on the **bounding box size** in the respective directions, in the BEV coordinates.

We tried tuning the magnitude of `scale_x` and `scale_y` by **altering the scalar factor** being multiplied with the `x_size` and `y_size` terms.

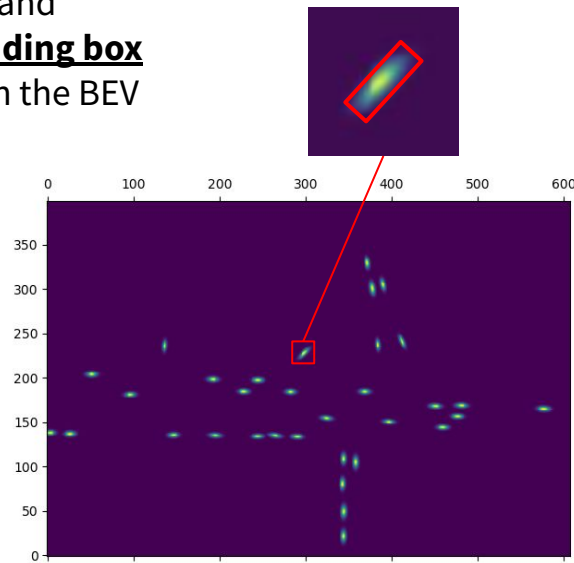


Figure 3: Target heatmap

Results

After training 5 epochs for each of the following parameter settings, we find that both experiment 1 and 2 shows a decreased average precision compared to that of Part A original model.

Experiment I: Original model + Focal loss

Alpha	Gamma	Average Precision	Average Precision (% change)
NAN (Part A AP)	NAN (Part A AP)	0.45	0%
.25	2	0.34	-24.4%
.25	1	0.27	-40%
.25	3	0.40	-11.1%
.10	3	0.25	-44.4%
.20	3	0.25	-44.4%

Experiment II: Original model + Rotated Anisotropic Gaussian

Scale X	Scale Y	Average Precision	Average Precision (% change)
NAN (Part A AP)	NAN (Part A AP)	0.45	0%
$2 * (\text{boxsize_x}^2) / \text{heatmap_norm_scale}$	$2 * (\text{boxsize_y}^2) / \text{heatmap_norm_scale}$	0.29	-35.6%
$1 * (\text{boxsize_x}^2) / \text{heatmap_norm_scale}$	$1 * (\text{boxsize_y}^2) / \text{heatmap_norm_scale}$	0.13	-71.1%

Results

The combined model also shows a lower average precision.

Experiment III: Original Model + Focal Loss + Anisotropic Gaussian

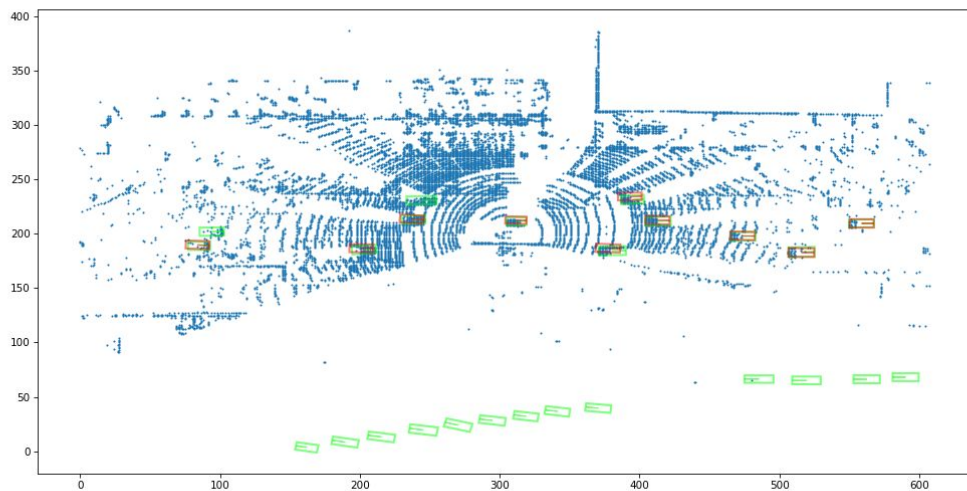


Figure 4: Detections (green: targets, red: predictions)

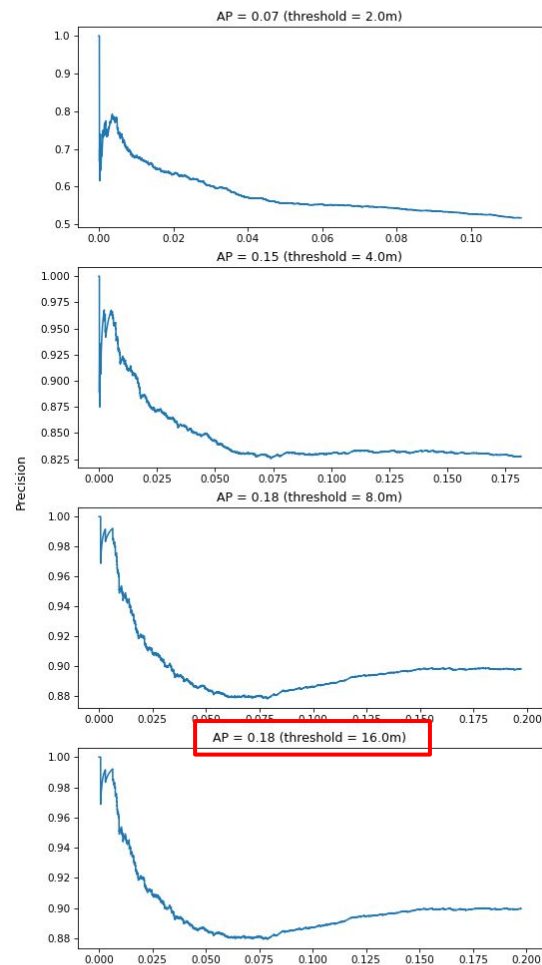


Figure 5: PR curves for different distance thresholds

Q & A

